

Projectree 포팅 매뉴얼

S15P11D205

Repository: lab.ssafy.com/s15-webmobile1-sub1/S15P11D205

작성일: 2026-08-09

Projectree 포팅 매뉴얼

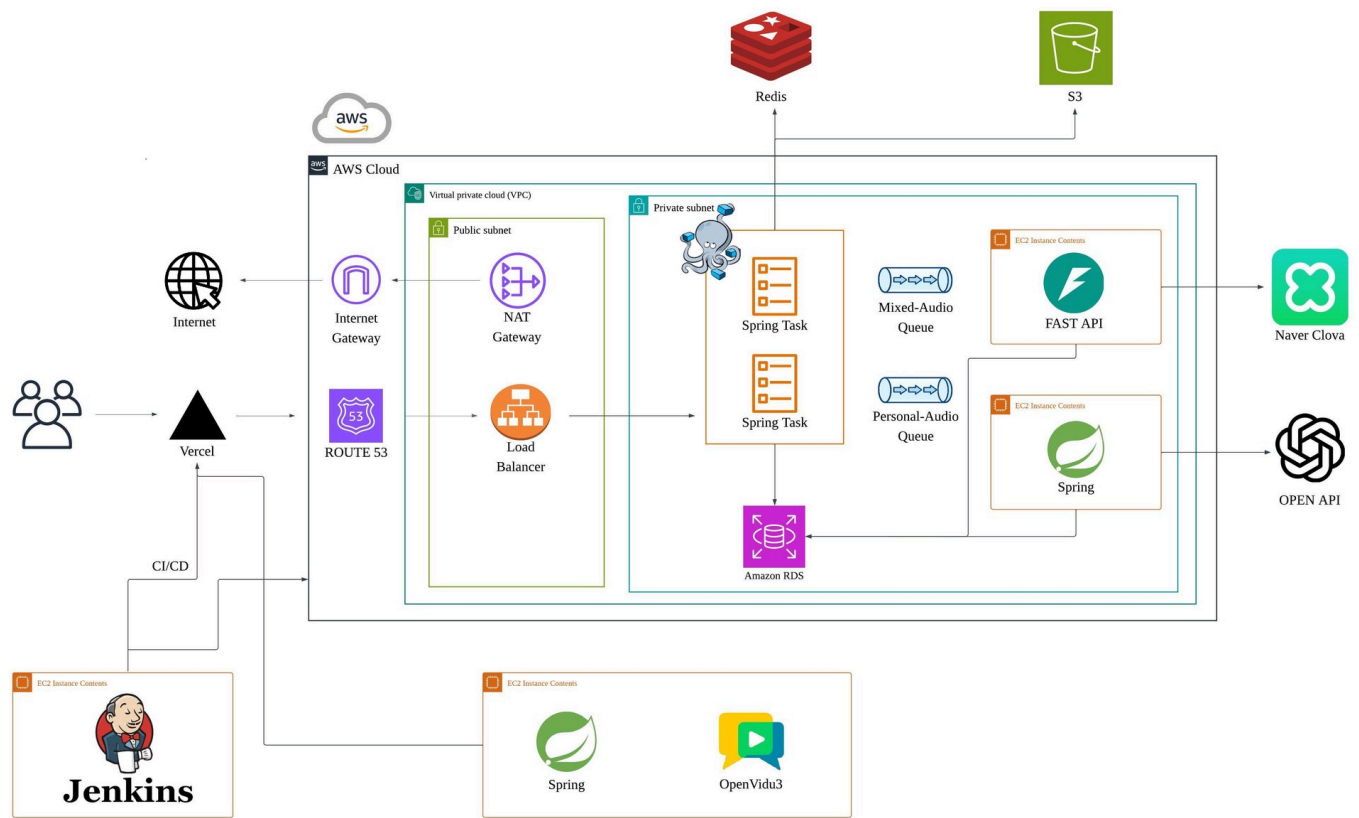
- 프로젝트명: Projectree (S15P11D205)
 - 최종 작성일: 2026-08-09
 - Repository: <https://lab.ssafy.com/s15-webmobile1-sub1/S15P11D205.git>
-

목차

1. 시스템 아키텍처
 2. 프로젝트 구성
 3. 개발/빌드 환경
 4. 빌드 시 사용되는 환경 변수
 5. 배포 시 특이사항
 6. DB 접속 정보 및 주요 프로퍼티 파일 목록
 7. CI/CD 파이프라인
 8. 외부 서비스 연동 정보
 9. 인프라 검증 / 모니터링 CLI 명령어
 10. 로컬 실행 방법
 11. 시연 시나리오
-

1. 시스템 아키텍처

1.1 전체 아키텍처 다이어그램



1.2 컴포넌트 요약

컴포넌트	역할
Route 53	도메인 라우팅
Vercel Edge Deployment	React 19 + Vite 8 SPA 배포, OpenVidu와 직접 통신
VPC / Public-Private Subnet	네트워크 격리
Internet Gateway / NAT Gateway	인터넷 송수신, Private Subnet 아웃바운드
ALB	HTTP(S) 로드밸런싱, ECS Task 라우팅
ECS Fargate (ssafy-spring-service)	Spring Boot API 서버, Task 2개 다중화
OpenVidu 3 미디어 서버	화상회의의 WebRTC 연결, 녹화본 S3 업로드
EC2 FastAPI Worker (data-pipeline)	STT 연동, 오디오 수집, 그래프 파이프라인
EC2 Spring AI Worker (personal-audio-backend)	개인 음성 분석, 회의록 요약
SQS Mixed-Audio / Personal-Audio Queue	비동기 오디오/분석 이벤트 큐
S3 (projectree-bucket)	녹음 파일, 그래프 스냅샷, 업로드 이미지 저장

컴포넌트	역할
Upstash Redis	세션 저장소
Amazon RDS	MySQL + PostgreSQL
EC2 Jenkins	빌드/배포 자동화

2. 프로젝트 구성

모듈 경로	스택	역할
Backend/	Spring Boot 4.1.0, Java 21	메인 API 서버
personal-audio-backend/	Spring Boot 4.1.0, Java 21	개인 음성 분석 AI Worker
data-pipeline/	Python 3.11, FastAPI	STT 및 그래프 파이프라인
Frontend/	React 19 + Vite 8, LiveKit Client SDK	사용자 웹 클라이언트

3. 개발/빌드 환경

3.1 Backend / personal-audio-backend

항목	값
언어	Java 21
빌드 도구	Gradle 9.5.1
프레임워크	Spring Boot 4.1.0
JVM	eclipse-temurin:21-jre
JDK 빌드 컨테이너	eclipse-temurin:21-jdk
WAS	Spring Boot 내장 Tomcat
IDE	IntelliJ IDEA Ultimate 2026.1.4
주요 의존성 (Backend)	spring-boot-starter-data-jpa, spring-boot-starter-data-redis, spring-boot-starter-session-data-redis, spring-boot-starter-mail, spring-boot-starter-validation, mysql-connector-j, spring-cloud-aws
주요 의존성 (personal-audio-backend)	software.amazon.awssdk:sqs, software.amazon.awssdk:s3, spring-boot-starter-webmvc

3.2 data-pipeline

항목	값
언어	Python 3.11
패키지 관리	setuptools
API 서버	FastAPI + Uvicorn
DB 마이그레이션	Alembic
주요 라이브러리	pydantic, sqlalchemy, alembic, psycpg, httpx, boto3, openai
실행 방식	venv 기반 EC2 직접 실행

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -e ".[dev]"
.venv\Scripts\python.exe -m alembic upgrade head
```

3.3 Frontend

항목	값
Node.js	22.x LTS
패키지 매니저	npm
언어	TypeScript 6.0.2
빌드 도구	Vite 8.1.1
UI 프레임워크	React 19.2.7
라우팅	React Router DOM 7.18.1
전역 상태관리	Zustand 5.0.14
화상회의 SDK	livekit-client 2.21.0
3D 시각화	React Three Fiber, Three.js
Lint	ESLint 10.6.0
배포	Vercel
모노레포/캐시	Turborepo Remote Cache

주요 라우트

경로	페이지 컴포넌트
/	Landing
/auth/:provider/callback	OAuthCallback
/invitations/:token	InvitationLanding
/home	Home
/projects	ProjectList
/projects/create	ProjectCreate
/mypage	MyPage
/projects/:projectId	ProjectHome
/projects/:projectId/members	ProjectMember
/projects/:projectId/meeting	ProjectMeeting
/projects/:projectId/tree	ProjectTree

4. 빌드 시 사용되는 환경 변수

4.1 Backend

분류	환경 변수	설명
DB	DB_HOST, DB_PORT, DB_NAME, DB_USERNAME, DB_PASSWORD	MySQL 8.0 접속 정보
Redis	REDIS_HOST, REDIS_PORT, REDIS_PASSWORD	Upstash Redis
AWS	AWS_ACCESS_KEY, AWS_SECRET_KEY, AWS_REGION	ECS Task Role로 위임되어 운영 환경에서는 미지정
Mail	MAIL_USERNAME, MAIL_PASSWORD	Gmail SMTP
OAuth	GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, NAVER_CLIENT_ID, NAVER_CLIENT_SECRET	소셜 로그인
CORS/Cookie	CORS_ALLOWED_ORIGINS, SESSION_COOKIE_SECURE, SESSION_COOKIE_SAME_SITE	운영 쿠키 정책
초대 메일	INVITATION_BASE_URL	프론트엔드 기본 URL
회의 동기화	MEETING_SYNC_ENABLED, MEETING_SYNC_FIXED_DELAY_MS, MEETING_SYNC_SCAN_COUNT, MEETING_SYNC_KEY_PREFIX	Redis 기반 회의 상태 동기화 스케줄러
회의 분석 그래프 스냅샷	MEETING_ANALYSIS_GRAPH_SNAPSHOT_MAX_SIZE_BYTES, MEETING_ANALYSIS_GRAPH_SNAPSHOT_S3_ENABLED, AWS_ANALYSIS_RESULT_BUCKET, MEETING_ANALYSIS_GRAPH_SNAPSHOT_S3_PREFIX, MEETING_ANALYSIS_GRAPH_SNAPSHOT_S3_ENDPOINT	그래프 스냅샷 저장 설정
회의 분석 Publisher	MEETING_ANALYSIS_PUBLISHER_ENABLED, MEETING_ANALYSIS_PUBLISHER_FIXED_DELAY_MS, MEETING_ANALYSIS_PUBLISHER_BATCH_SIZE, MEETING_ANALYSIS_PUBLISHER_MAX_ATTEMPTS, MEETING_ANALYSIS_PUBLISHER_LEASE_SECONDS, MEETING_ANALYSIS_COMMAND_QUEUE_URL	Command 발행, Outbox to SQS
회의 분석 Result Consumer	MEETING_ANALYSIS_RESULT_CONSUMER_ENABLED, AWS_ANALYSIS_RESULT_QUEUE_URL, MEETING_ANALYSIS_RESULT_WAIT_TIME_SECONDS, MEETING_ANALYSIS_RESULT_MAX_NUMBER_OF_MESSAGES	Python Worker 결과 수신
회의록 Callback	MEETING_RECORD_CALLBACK_API_KEY	data-pipeline과 공유하는 비밀값
S3	AWS_S3_BUCKET	파일 저장 버킷

4.2 personal-audio-backend

분류	환경 변수	설명
DB	DB_HOST, DB_PORT, DB_NAME, DB_USERNAME, DB_PASSWORD, JPA_DDL_AUTO	MySQL 8.0
SQS	AWS_SQS_ENABLED, SQS_QUEUE_NAME, SQS_MAX_CONCURRENT_MESSAGES	Personal-Audio Queue 리스너
S3	S3_BUCKET	개인 음성 원본/결과 저장
오디오 처리	AUDIO_SILENCE_THRESHOLD, AUDIO_MIN_SILENCE_DURATION, AUDIO_ANALYZE_TIMEOUT	ffmpeg 무음 구간 분석
AI (OpenAI/GMS)	GMS_BASE_URL, GMS_API_KEY, GMS_MODEL, GMS_TRANSCRIBE_MODEL, GMS_MAX_UPLOAD_BYTES, GMS_TIMEOUT	OpenAI 호환 게이트웨이
AI (오디오)	AI_AUDIO_MAX_SECONDS, AI_AUDIO_CONVERT_TIMEOUT	오디오 처리 제한
서버	SERVER_PORT	서버 포트

4.3 data-pipeline

분류	환경 변수	설명
DB	DATABASE_URL, DB_POOL_SIZE, DB_MAX_OVERFLOW, DB_POOL_TIMEOUT_SECONDS, DB_POOL_RECYCLE_SECONDS, DB_CONNECT_TIMEOUT_SECONDS, DB_STATEMENT_TIMEOUT_MS	PostgreSQL, pgvector, 그래프 정보
API	API_HOST, API_PORT, API_MAX_REQUEST_BODY_BYTES, API_GRACEFUL_SHUTDOWN_SECONDS, INTERNAL_API_TOKEN	내부 API 인증
STT	STT_ADAPTER, CLOVA_INVOKE_URL, CLOVA_SECRET, CLOVA_TIMEOUT_SECONDS	Naver Clova STT
S3/SQS	AWS_REGION, SQS_QUEUE_URL, RECORDING_READY_QUEUE_URL, ANALYSIS_COMMAND_QUEUE_URL, S3_ALLOWED_BUCKETS, S3_INPUT_PREFIX, S3_ALLOWED_EXTENSIONS, S3_MAX_AUDIO_BYTES	오디오 입수 경로
LLM/임베딩	LLM_ADAPTER, EMBEDDING_ADAPTER, B_MODEL_ADAPTER, GMS_KEY, OPENAI_BASE_URL, OPENAI_MODEL, NO_EXTERNAL_AI_CALLS	LLM 연동
Outbox/결과	OUTBOX_TRANSPORT, PROJECTREE_ANALYSIS_RESULT_QUEUE_URL, PROJECTREE_ANALYSIS_RESULT_QUEUE_TYPE, AWS_S3_BUCKET, PROJECTREE_GRAPH_SNAPSHOT_PREFIX, PROJECTREE_GRAPH_SNAPSHOT_MAX_SIZE_BYTES, SUMMARY_ADAPTER	Backend로 결과 회신
Java Callback	JAVA_BASE_URL, MEETING_RECORD_CALLBACK_API_KEY, MEETING_RECORD_CALLBACK_TIMEOUT_SECONDS	Backend와 공유 비밀값
OpenVidu 수집	OPENVIDU_RECORDING_BUCKET, OPENVIDU_RECORDING_PREFIX, OPENVIDU_SUPPORTED_KINDS	녹화본 수집 경로

4.4 Frontend

환경 변수	필수 여부	설명
VITE_BASE_URL	필수	Backend REST API 및 SSE 기본 URL
VITE_DEV_BASE_URL	개발 전용	로컬 프록시 target
VITE_GOOGLE_CLIENT_ID	필수	Google OAuth 클라이언트 ID
VITE_NAVER_CLIENT_ID	필수	Naver OAuth 클라이언트 ID
VITE_OPENVIDU_URL	필수	OpenVidu 미디어 서버 WebSocket URL

5. 배포 시 특이사항

1. 모든 모듈이 별도의 `application-prod.yml` 없이 단일 `application.yaml` 에서 환경변수를 주입받는 방식으로 구성되어 있어, 운영과 로컬 환경은 ECS Task Definition 또는 `.env` 에 설정된 환경변수 값으로 구분된다.
 2. ECS Fargate의 Task Role을 통해 AWS 권한을 위임받는 구조라서, `AWS_ACCESS_KEY` / `AWS_SECRET_KEY` 는 운영 환경에서 별도로 하드코딩하지 않고 Backend 배포 시 비워두면 된다.
 3. Redis(Upstash) 연결은 TLS 사용을 전제로 코드가 작성되어 있어(`spring.data.redis.ssl.enabled: true`), Upstash 콘솔에서 발급받은 TLS 접속 정보를 그대로 사용하면 된다.
 4. 그래프 스냅샷 관련 S3 설정은 `MEETING_ANALYSIS_GRAPH_SNAPSHOT_S3_BUCKET` 과 `MEETING_ANALYSIS_GRAPH_SNAPSHOT_S3_REGION` 이 함께 등록되어야 정상적으로 인식된다.
 5. 회의록 Callback 인증을 위해 Backend와 data-pipeline이 동일한 `MEETING_RECORD_CALLBACK_API_KEY` 값을 공유하도록 맞춰두었다.
 6. `data-pipeline` 은 별도 컨테이너화 없이, EC2에서 `venv` 환경으로 `python -m data_pipeline. {api|worker|outbox_publisher}` 세 프로세스를 각각 기동하는 방식으로 운영된다.
 7. `data-pipeline`이 사용하는 PostgreSQL은 pgvector 확장을 사용하므로, 배포 전 `CREATE EXTENSION IF NOT EXISTS vector;` 를 실행한 뒤 `alembic upgrade head` 로 스키마를 최신화한다.
 8. OpenVidu 녹음 수집 과정에서 Worker가 `s3://projecttree-bucket/meetings/*` 객체에 접근하므로, 해당 경로에 대한 `s3:GetObject` 권한을 부여해두었다.
 9. Jenkins를 통한 자동 빌드/배포는 현재 Backend 모듈을 대상으로 구성되어 있다. GitLab `Develop_BE` 브랜치를 체크아웃한 뒤 `dir('Backend')` 에서 Gradle 빌드를 수행하며, 나머지 모듈은 별도 Job이나 수동 배포로 운영되고 있다.
 10. Frontend는 ECS/EC2와 별개로 Vercel Edge에 배포되는 구조다. Vercel 자체 CI가 빌드/배포를 담당하고, Jenkins는 필요할 때 `vercel --prod` CLI를 트리거하는 역할만 한다.
 11. 화상회의 트래픽은 Backend를 거치지 않는다. Backend는 LiveKit 참여 토큰과 `livekitUrl` 발급까지만 담당하고, 이후 프론트엔드가 OpenVidu 3 미디어 서버와 직접 WebRTC로 연결된다.
 12. `MeetingSummaryModal` 과 `MeetingNodeReviewModal` 컴포넌트는 코드상으로는 구현되어 있지만, 아직 어떤 페이지에도 연결되어 있지 않은 상태다. 그래서 "AI 회의록 검토"."AI 노드 분류 검토" 화면은 현재 사용자 플로우에서는 열리지 않는다.
-

6. DB 접속 정보 및 주요 프로퍼티 파일 목록

파일	설명
application.yaml	Backend DB/Redis/AWS/Mail/OAuth 설정 정의
.env.example	Backend 로컬 개발용 환경변수 템플릿
.env	Backend 실제 환경변수 값
docker-compose.yml	Backend 로컬 MySQL 컨테이너 정의
application.yaml	personal-audio-backend DB/SQS/S3/AI 설정 정의
docker-compose.yml	personal-audio-backend 로컬 MySQL + 앱 컨테이너
.env.example	data-pipeline PostgreSQL/STT/LLM/SQS 설정 템플릿
docker-compose.yml	data-pipeline 로컬 PostgreSQL + pgvector 컨테이너
alembic.ini	data-pipeline 마이그레이션 설정
.env.local / .env.check / .env.vercel-check	Frontend Vercel 연동 및 VITE_* 환경변수

6.1 DB 접속 정보 요약

DB	엔진	사용 모듈
Amazon RDS (MySQL 8.0)	MySQL	Backend, personal-audio-backend
Amazon RDS (PostgreSQL, pgvector)	PostgreSQL	data-pipeline
Upstash Redis	Redis (TLS)	Backend

6.2 DB 덤프 파일 및 복원 가이드

① MySQL 8.0 (Backend, personal-audio-backend 공용 RDS)

```
mysqldump -h <DB_HOST> -P 3306 -u <DB_USERNAME> -p \
  --databases projectree --single-transaction --routines --triggers \
  > exec/ssafydb_dump.sql

mysql -h <DB_HOST> -P 3306 -u <DB_USERNAME> -p < exec/ssafydb_dump.sql

docker exec -i projectree-mysql mysql -uroot -p"${MYSQL_ROOT_PASSWORD}" projectree <
exec/ssafydb_dump.sql
```

② PostgreSQL 16 + pgvector (data-pipeline 그래프 정보 RDS)

```
pg_dump "postgresql://pipeline:<PASSWORD>@<DB_HOST>:5432/pipeline" \  
--format=plain --no-owner --no-privileges \  
> exec/ssafydb_pg_dump.sql
```

```
psql "postgresql://pipeline:<PASSWORD>@<DB_HOST>:5432/pipeline" \  
-c "CREATE EXTENSION IF NOT EXISTS vector;"
```

```
psql "postgresql://pipeline:<PASSWORD>@<DB_HOST>:5432/pipeline" \  
-f exec/ssafydb_pg_dump.sql
```

```
.venv\Scripts\python.exe -m alembic current
```

```
.venv\Scripts\python.exe -m alembic check
```

7. CI/CD 파이프라인

7.1 배포 흐름



7.2 Jenkinsfile

항목	값
Region	ap-northeast-2
ECR Repository	ssafy-spring-backend
ECS Cluster	ssafy-main-cluster2
ECS Service	ssafy-spring-service
GitLab Branch	Develop_BE
GitLab Credential	gitlab-git-credentials

```

pipeline {
    agent any

    environment {
        AWS_REGION      = 'ap-northeast-2'
        ECR_REPOSITORY = 'ssafy-spring-backend'
        CONTAINER_NAME = 'ssafy-spring-container'
        GITLAB_CREDENTIAL_ID = 'gitlab-git-credentials'
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scmGit(
                    branches: [[name: '*/Develop_BE']],
                    userRemoteConfigs: [[
                        credentialsId: "${GITLAB_CREDENTIAL_ID}",
                        url: 'https://lab.ssafy.com/s15-webmobile1-sub1/S15P11D205.git'
                    ]]
                )
            }
        }

        stage('Gradle Build') {
            steps {
                dir('Backend') {
                    sh 'chmod +x ./gradlew'
                    sh './gradlew clean bootJar -x test'
                }
            }
        }

        stage('Docker Image Build and ECR Push') {
            steps {
                dir('Backend') {
                    script {
                        def accountId = sh(script: 'aws sts get-caller-identity --query Account --output text', returnStdout: true).trim()
                        sh "aws ecr get-login-password --region ${AWS_REGION} | docker login --username AWS --password-stdin ${accountId}.dkr.ecr.${AWS_REGION}.amazonaws.com"

                        def imageTag =
"${accountId}.dkr.ecr.${AWS_REGION}.amazonaws.com/${ECR_REPOSITORY}:${BUILD_NUMBER}"
                        def latestTag =
"${accountId}.dkr.ecr.${AWS_REGION}.amazonaws.com/${ECR_REPOSITORY}:latest"

                        sh "docker build -t ${imageTag} -t ${latestTag} ."
                        sh "docker push ${imageTag}"
                        sh "docker push ${latestTag}"
                    }
                }
            }
        }
    }
}

```



```

    }
}

stage('Deploy to ECS') {
    steps {
        sh "aws ecs update-service --cluster ssafy-main-cluster2 --service ssafy-spring-service -
-force-new-deployment --region ${AWS_REGION}"
    }
}
}
}
}

```

7.3 Dockerfile

Backend/Dockerfile

```

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY ./build/libs/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

```

personal-audio-backend/Dockerfile

```

FROM eclipse-temurin:21-jdk AS build
WORKDIR /workspace
COPY gradlew settings.gradle build.gradle ./
COPY gradle gradle
RUN chmod +x gradlew && ./gradlew --no-daemon dependencies
COPY src src
RUN ./gradlew --no-daemon clean bootJar -x test

FROM eclipse-temurin:21-jre
WORKDIR /app
RUN apt-get update \
&& apt-get install -y --no-install-recommends ffmpeg \
&& rm -rf /var/lib/apt/lists/*
COPY --from=build /workspace/build/libs/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

```

7.4 Vercel 배포

```

npm i -g vercel
vercel pull --yes --environment=production --token=$VERCEL_TOKEN
vercel build --prod --token=$VERCEL_TOKEN
vercel deploy --prebuilt --prod --token=$VERCEL_TOKEN

```

7.5 Jenkins Credentials

Credential ID	타입	용도
gitlab-git-credentials	Username with password	GitLab Develop_BE 브랜치 체크아웃

AWS 인증은 별도 Jenkins Credential 없이, Jenkins가 실행되는 EC2에 부여된 IAM Instance Profile을 통해 처리되고 있다. Vercel 배포에 쓰이는 `VERCEL_TOKEN` 은 Jenkins Global Credentials에서 별도로 관리되고 있다.

8. 외부 서비스 연동 정보

8.0 요약

서비스	용도	연동 모듈
Google OAuth	소셜 로그인	Backend, Frontend
Naver OAuth	소셜 로그인	Backend, Frontend
Naver Clova STT	음성 텍스트 변환	data-pipeline
OpenAI API (GMS 게이트웨이)	회의록 요약, 임베딩, LLM 분석	personal-audio-backend, data-pipeline
Gmail SMTP	프로젝트 초대 메일 발송	Backend
Upstash Redis	세션 스토어	Backend
OpenVidu 3 (LiveKit)	화상회의 미디어 서버	Frontend
Vercel	프론트엔드 호스팅	Frontend

8.1 Google OAuth

항목	내용
콘솔 위치	Google Cloud Console → API 및 서비스 → 사용자 인증 정보
발급 절차	프로젝트 생성 → OAuth 동의 화면 구성 → OAuth 클라이언트 ID 생성 → 리디렉션 URI 등록
등록한 설정값	리디렉션 URI: <code>{origin}/auth/google/callback</code>
대응 환경변수	Backend: GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET / Frontend: VITE_GOOGLE_CLIENT_ID

8.2 Naver OAuth

항목	내용
콘솔 위치	Naver Developers
발급 절차	애플리케이션 등록 → 네이버 로그인 API 추가 → 서비스 URL/Callback URL 등록 → Client ID/Secret 확인
등록한 설정값	Callback URL: <code>{origin}/auth/naver/callback</code>
대응 환경변수	Backend: NAVER_CLIENT_ID, NAVER_CLIENT_SECRET / Frontend: VITE_NAVER_CLIENT_ID

8.3 Naver Clova STT

항목	내용
콘솔 위치	NAVER Cloud Platform Console → CLOVA Speech Recognition
발급 절차	계정/결제수단 등록 → CSR 서비스 신청 → 도메인 생성 후 Invoke URL 발급 → Secret Key 발급

항목	내용
등록한 설정값	STT_ADAPTER=clova
대응 환경변수	data-pipeline: CLOVA_INVOKE_URL, CLOVA_SECRET, CLOVA_TIMEOUT_SECONDS

8.4 OpenAI API (GMS 게이트웨이)

항목	내용
콘솔 위치	SSAFY GMS 게이트웨이 관리 콘솔
발급 절차	GMS 콘솔에서 API Key 발급 신청 → 각 모듈 환경변수에 등록
등록한 설정값	Base URL: <code>https://gms.ssafy.io/gmsapi/api.openai.com/v1</code>
대응 환경변수	personal-audio-backend: GMS_BASE_URL, GMS_API_KEY, GMS_MODEL, GMS_TRANSCRIBE_MODEL / data-pipeline: GMS_KEY, OPENAI_BASE_URL, OPENAI_MODEL

8.5 Gmail SMTP

항목	내용
콘솔 위치	Google 계정 관리 → 보안 → 앱 비밀번호
발급 절차	2단계 인증 활성화 → 앱 비밀번호 생성 → MAIL_PASSWORD로 사용
등록한 설정값	SMTP smtp.gmail.com:587, STARTTLS
대응 환경변수	Backend: MAIL_USERNAME, MAIL_PASSWORD

8.6 Upstash Redis

항목	내용
콘솔 위치	Upstash Console → Redis
발급 절차	Redis 데이터베이스 생성 → Endpoint/Port/Password 확인
등록한 설정값	TLS 포트(redis://, 6379) 접속
대응 환경변수	Backend: REDIS_HOST, REDIS_PORT, REDIS_PASSWORD

8.7 OpenVidu 3 (LiveKit)

항목	내용
콘솔 위치	별도 프로비저닝된 WebRTC 미디어 서버 자체 운영
발급 절차	Backend가 회의 참여 시점에 LiveKit 서버 API로 참여 토큰과 livekitUrl을 발급
등록한 설정값	녹화본은 S3 projectree-bucket의 meetings/ prefix로 업로드
대응 환경변수	Frontend: VITE_OPENVIDU_URL

8.8 Vercel

항목	내용
콘솔 위치	Vercel Dashboard → 프로젝트 Settings
발급 절차	GitLab 연동으로 프로젝트 Import → 환경변수 등록 → CI용 Personal Access Token 발급
등록한 설정값	vercel.json SPA rewrite 설정, Turborepo Remote Cache 사용
대응 환경 변수	Jenkins/CI: VERCEL_TOKEN / Frontend 빌드: VITE_BASE_URL, VITE_GOOGLE_CLIENT_ID, VITE_NAVER_CLIENT_ID, VITE_OPENVIDU_URL

9. 인프라 검증 / 모니터링 CLI 명령어

```
# ECS 서비스 상태 확인
aws ecs describe-services \
  --cluster ssafy-main-cluster2 \
  --services ssafy-spring-service \
  --region ap-northeast-2

# 실행 중인 Task 상세 확인
aws ecs list-tasks --cluster ssafy-main-cluster2 --service-name ssafy-spring-service --region ap-northeast-2

aws ecs describe-tasks \
  --cluster ssafy-main-cluster2 \
  --tasks <TASK_ARN> \
  --region ap-northeast-2

# SQS 큐 상태 확인
aws sqs get-queue-attributes \
  --queue-url "<MIXED_AUDIO_QUEUE_URL>" \
  --attribute-names QueueArn VisibilityTimeout \
    ApproximateNumberOfMessages \
    ApproximateNumberOfMessagesNotVisible \
    RedrivePolicy

# CloudWatch Logs 실시간 tail
aws logs tail /ecs/ssafy-spring-service --follow --region ap-northeast-2

# ALB 타겟 헬스체크 확인
aws elbv2 describe-target-health --target-group-arn <TARGET_GROUP_ARN>

# RDS 인스턴스 상태 확인
aws rds describe-db-instances --db-instance-identifier <DB_INSTANCE_ID>

# S3 버킷 내 최근 업로드 오브젝트 확인
aws s3 ls s3://projectree-bucket/meetings/ --recursive | tail -n 20
```

10. 로컬 실행 방법

10.1 Backend

```
cd Backend
cp .env.example .env
docker compose up -d
./gradlew bootRun
```

10.2 personal-audio-backend

```
cd personal-audio-backend
cp .env.example .env
docker compose up -d
./gradlew bootRun
```

10.3 data-pipeline

```
cd data-pipeline
python -m venv .venv
.venv\Scripts\python.exe -m pip install -e ".[dev]"
docker compose up -d
.venv\Scripts\python.exe -m alembic upgrade head
.venv\Scripts\python.exe -m data_pipeline.api
.venv\Scripts\python.exe -m data_pipeline.worker
.venv\Scripts\python.exe -m data_pipeline.outbox_publisher
```

10.4 Frontend

```
cd Frontend
vercel link
vercel env pull .env.local
docker compose up -d
```

Frontend/docker-compose.yml 의 `react-vite` 서비스(`node:22-alpine` 기반)가 `npm install && npm run dev` 를 컨테이너 안에서 실행하며, `5173` 포트로 매핑되어 `http://localhost:5173` 에서 접속할 수 있다. 소스 코드 변경 시 HMR(Hot Module Replacement)도 그대로 동작한다.

10.5 전체 서비스 기동 및 복구 순서

1. DB / Redis 기동 → 2. DB 스키마 마이그레이션 → 3. AI Worker 기동 → 4. Core Backend 기동 → 5. Frontend 배포/기동

단계	대상	확인 방법
1. DB / Redis 기동	Amazon RDS, Upstash Redis	<code>aws rds describe-db-instances --db-instance-identifier <DB_INSTANCE_ID></code>
2. DB 스키마 마이그레이션	data-pipeline PostgreSQL	<code>CREATE EXTENSION IF NOT EXISTS vector;</code> 실행 후 <code>alembic upgrade head</code>
3. AI Worker 기동	FastAPI Worker, Spring AI Worker	DB가 준비된 이후 기동
4. Core Backend 기동	ECS Fargate ssafy-spring-service	<code>aws ecs update-service --force-new-deployment</code>
5. Frontend 배포/기동	Vercel, npm run dev	Backend와 OpenVidu가 응답 가능한 상태에서 마지막으로 배포

11. 시연 시나리오

Projectree의 시연은 소셜 로그인부터 화상회의, 회의 종료 후 AI 분석 결과 확인까지 이어지는 전체 흐름을 보여주는 것을 목표로 하며, 총 소요 시간은 약 10~15분이다. 로그인은 Google/Naver OAuth 전용이라 별도 시드 계정이 없으므로, 발표자 개인 계정으로 사전 로그인 테스트를 마쳐두면 된다.

11.1 사전 준비 체크리스트

항목	확인 내용
서비스 상태	ECS Task, Worker 프로세스 정상 기동 확인
프론트엔드	projectree.site 정상 접속 확인
브라우저	시크릿 창 2개(발표자/참가자용) 준비, 마이크·카메라 권한 허용, 하울링 방지를 위해 한쪽은 음소거
네트워크	WebRTC 아웃바운드 트래픽 확인
테스트 데이터	시연용 프로젝트/회의방 초기화
백업 화면	회의록·노드 생성이 끝난 백업 프로젝트 준비

11.2 소셜 로그인

단계	화면	동작	설명
1	랜딩 페이지	헤더 로그인 또는 Get Started 클릭	로그인 모달 오픈
2	로그인 모달	Google로 계속하기 클릭	Google 인증 화면으로 이동
3	Google 동의 화면	계정 선택 후 동의	인가 코드와 함께 콜백으로 리다이렉트
4	콜백 처리 화면	자동 처리	Backend가 토큰 교환과 세션 생성을 수행
5	/home	자동 이동	로그인 완료, 프로젝트 목록 표시

프론트엔드는 인가 코드만 Backend로 전달하고, 실제 토큰 교환과 세션 발급은 Backend가 담당한다. 세션은 Upstash Redis에 저장된다.

11.3 화상회의

단계	화면	동작	설명
1	홈/프로젝트 목록	프로젝트 선택 또는 새로 생성	프로젝트가 없으면 생성 폼에서 먼저 생성
2	프로젝트 사이드바	회의 메뉴 클릭	회의 페이지로 이동
3	회의 참여 모달	장치 확인 후 참여 클릭	Backend가 LiveKit 참여 토큰 발급
4	연결 중 화면	자동 진행	프론트엔드가 미디어 서버에 직접 연결
5	화상회의 화면	카메라/마이크 확인	녹음 중 배지, 경과 시간, 참가자 수 표시

단계	화면	동작	설명
6	두 번째 참가자	동일하게 입장	참가자 간 영상 타일 상호 표시 확인

화상회의는 Backend가 참여 토큰만 발급하고, 실제 오디오/비디오는 프론트엔드가 미디어 서버와 직접 주고 받는 구조다.

11.4 회의 종료와 분석 결과 확인

회의 중 화면에는 녹음 중 배지, 경과 시간, 참가자 수만 표시되며, 회의록과 그래프 노드는 회의 종료 후 비동기로 생성된다.

단계	화면	동작	설명
1	컨트롤 바	회의 종료 클릭	종료 확인 모달 오픈
2	종료 모달	회의록 생성 / 노드 생성 선택 후 종료	선택한 항목만 분석 요청됨
3	프로젝트 홈	자동 이동 후 잠시 뒤 새로고침	최근 회의록, AI 피드백, 발화 비율 반영 확인
4	노드 메뉴	/projects/:id/tree 진입	3D 그래프에 새 노드 반영 확인

STT는 Naver Clova, 요약과 분석은 OpenAI를 사용하며, 결과는 프로젝트 홈과 노드 화면에서 확인한다. "AI 회의록 검토"·"AI 노드 분류 검토" 모달은 아직 화면에 연결되어 있지 않아 이번 시연에서는 다루지 않는다. 분석 결과가 1분 이상 반영되지 않으면 새로고침을 반복하기보다, 미리 준비해 둔 백업 프로젝트 화면으로 자연스럽게 전환해 시연을 이어가면 된다.

11.5 시연 마무리

소셜 로그인부터 화상회의, 회의 종료 후 자동 분석까지 하나의 흐름으로 정리하며 마무리하고, 실시간 데모가 원활하지 않을 경우를 대비해 사전 녹화 영상이나 스크린샷을 백업으로 준비해 둔다.